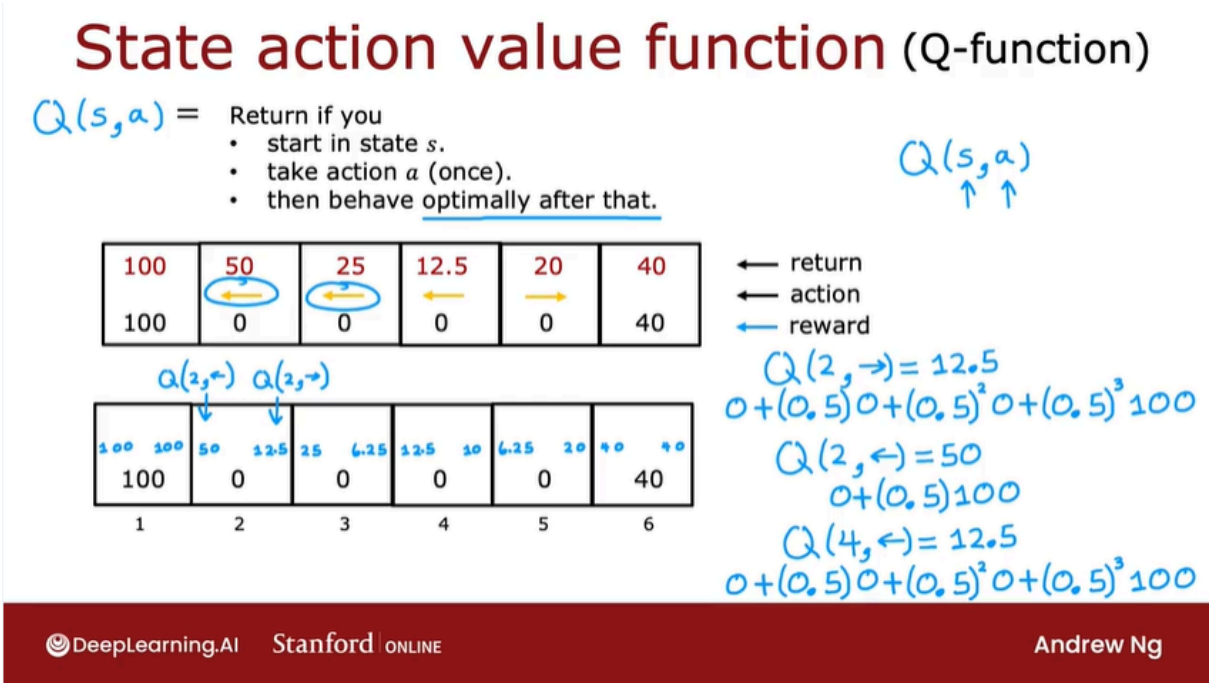
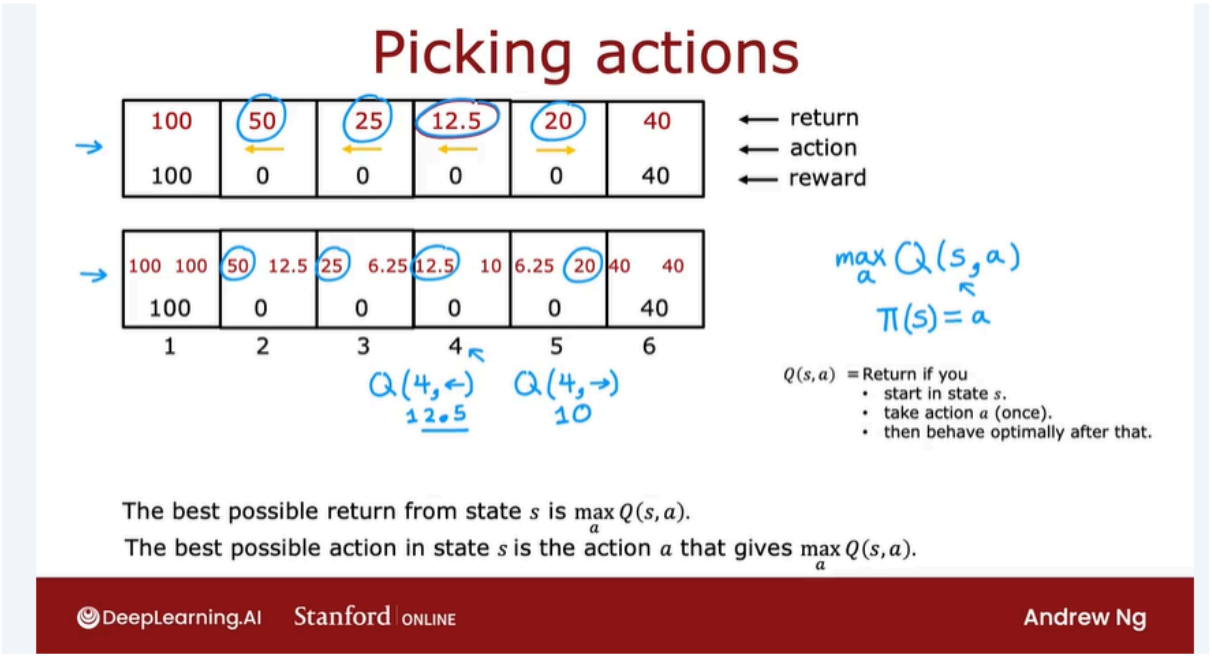
State-action value definition



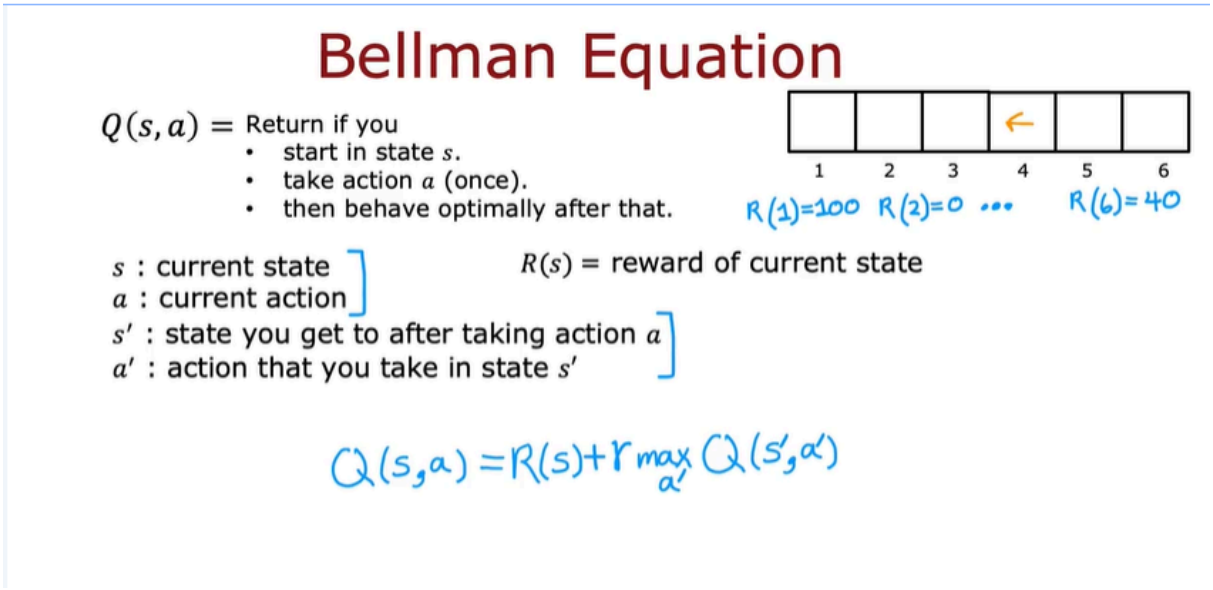


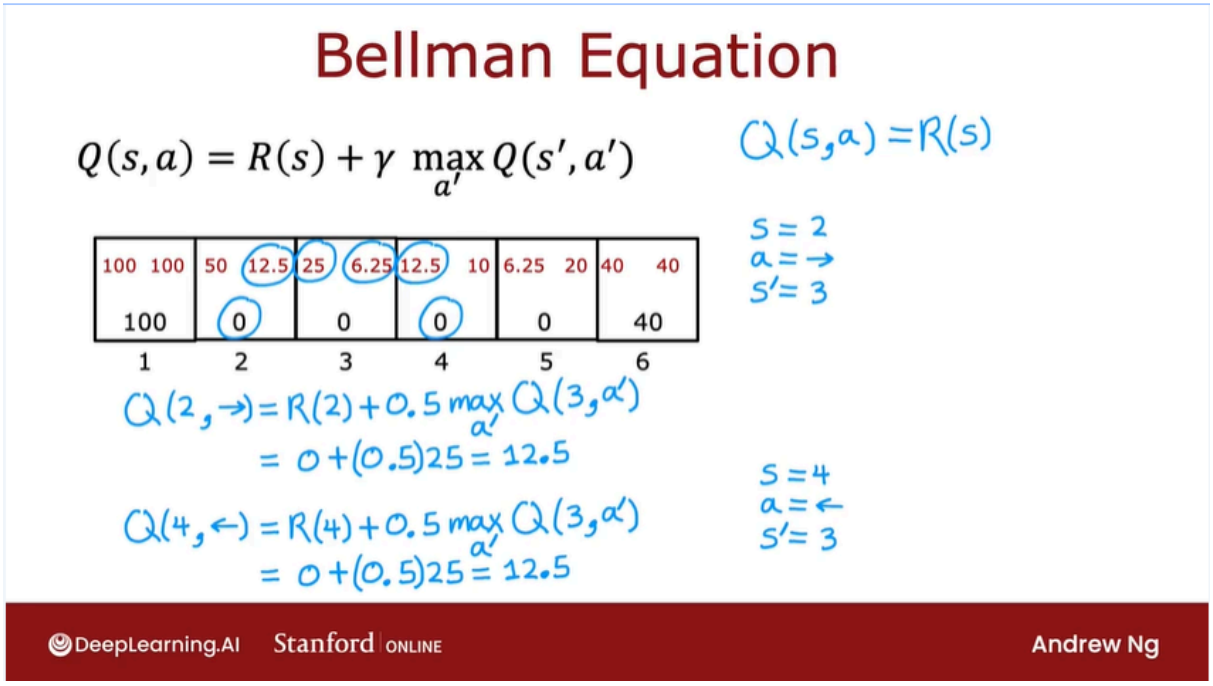
Q*: optimal Q function

And this Q function is sometimes also called the optimal Q function. These terms just refer to the Q function exactly as we've defined it. So if you look at the reinforcement learning literature and read about Q*or the Q function, that just means the state action value function that we've been talking about.

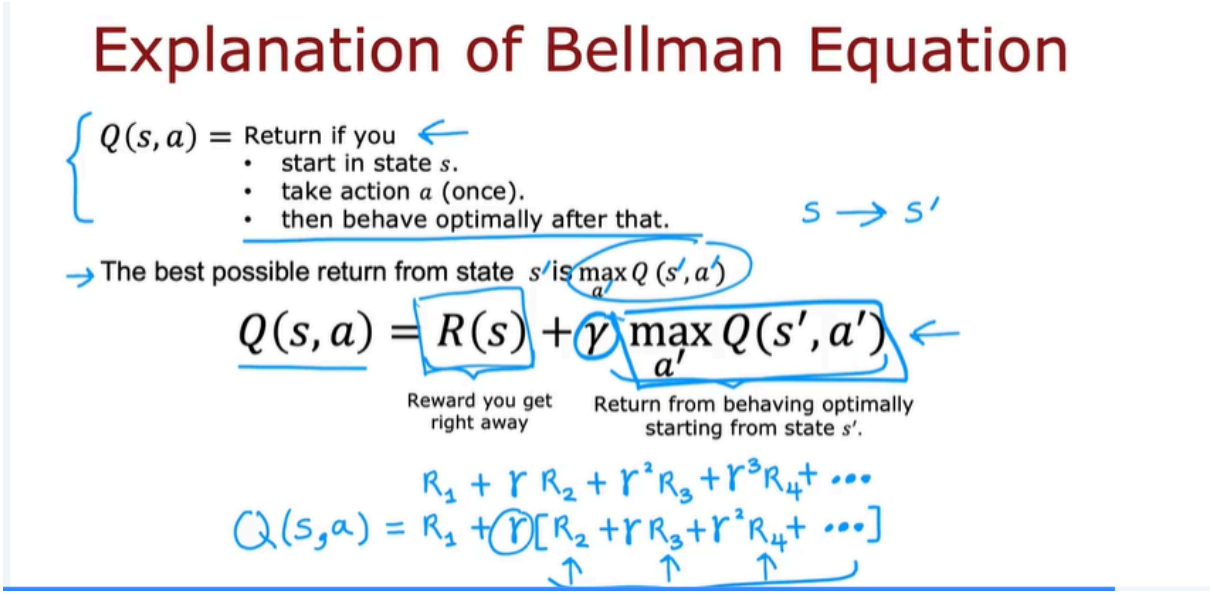
So to summarize if you can compute Q(s,a). For every state and every action, then that gives us a good way to compute the optimal policy $\pi(s)$.

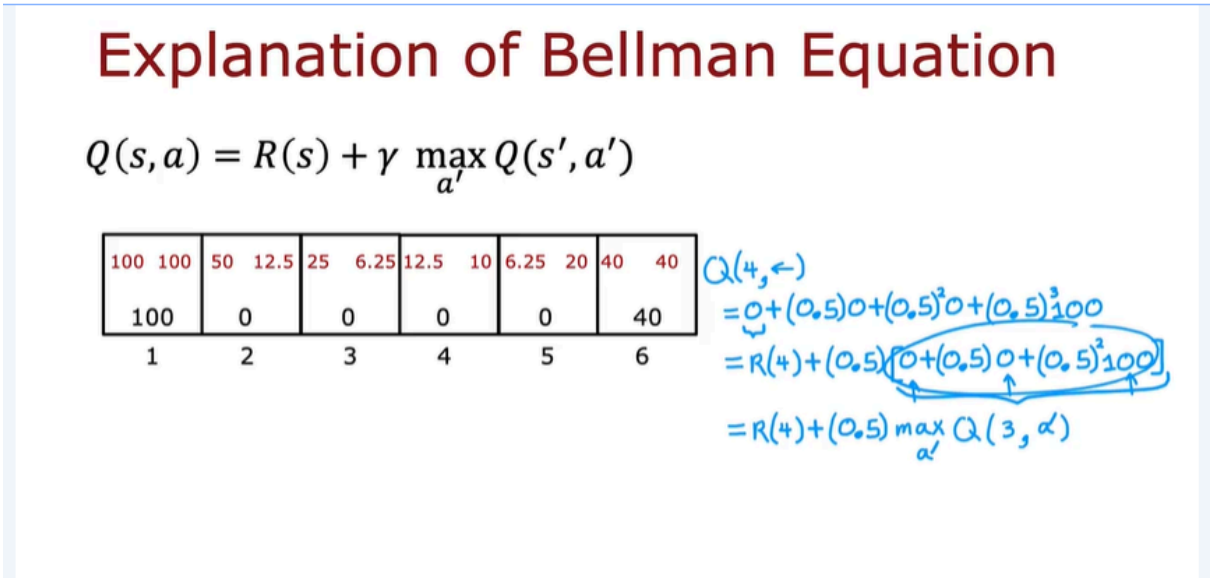
Bellman Equation





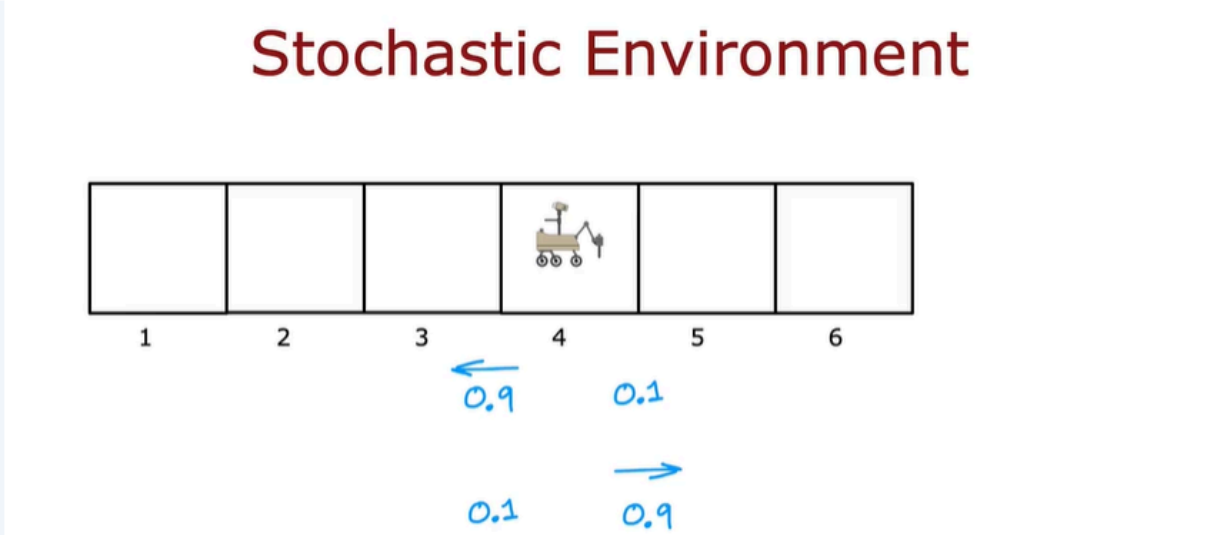
if you're in a terminal state, then Bellman Equation simplifies to Q(s, a) = R(s) because there's no state s′ and so that second term would go away.





Random (stochastic) environment

Stochastic Environment

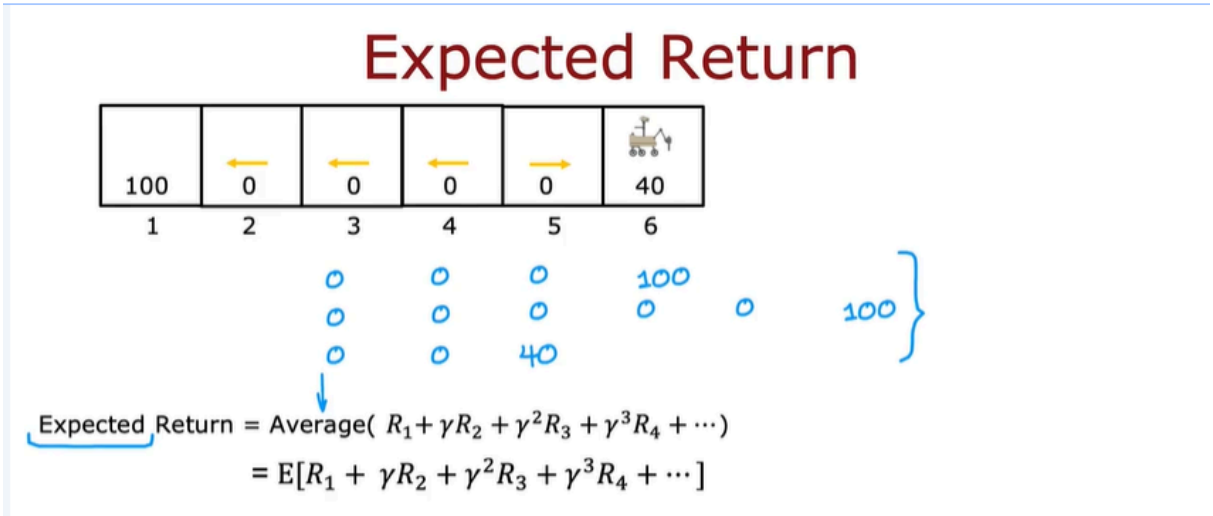


In this video, we'll talk about how these reinforcement learning problems work, continuing with our simplifying Mars Rover example, let's say you take the action and command it to go left.

Most of the time you'll succeed but what if 10 percent of the time or 0.1 of the time, it actually ends up accidentally slipping and going in the opposite direction? If you command it to go left, it has a 90 percent chance or 0.9 chance of correctly going in the left direction.

But the 0.1 chance of actually heading to the right so that it has a 90 percent chance of ending up in state three in this example and a 10 percent chance of ending up in state five.

Conversely, if you were to command it to go right and take the action, right, it has a 0.9 chance of ending up in state five and 0.1 chance of ending up in state three. This would be an example of a stochastic environment.



Let's see what happens in this reinforcement learning problem. Let's say you use this policy shown here, where you go left in states 2 3 4 and go rights or try to go right in state five.

If you were to start in state four and you were to follow this policy, then the actual sequence of states you visit may be random.

For example, in state four, you will go left, and maybe you're a little bit lucky, and it actually gets the state three, and then you try to go left again, and maybe it actually gets there. You tell it to go left again, and it gets to that state. If this is what happens, you end up with the sequence of rewards 0 → 0 → 0 → 100.

But if you were to try this exact same policy a second time, maybe you're a little less lucky, the second time you start here. Try to go left and say it succeeds so a zero from state four → zero from state three, here you tell it to go left, but you've got unlucky this time and the robot slips and ends up heading back to state four instead. Then you tell it to go left, and left, and left, and eventually get to that reward of 100.

In that case, this will be the sequence of rewards you observe. This one from four to three to four three two then one, or is even possible, if you tell from state four to go left following the policy you may get unlucky even on the first

step and you end up going to state five because it slipped. Then state five, you command it to go right, and it succeeds as you end up here. In this case, the sequence of rewards you see will be 0 → 0 → 40, because it went from four to five, and then states six, we had previously written out the return as this sum of discounted rewards.

But when the reinforcement learning problem is stochastic, there isn't one sequence of rewards that you see for sure instead you see this sequence of different rewards. In a stochastic reinforcement learning problem, what we're interested in is not maximizing the return because that's a random number.

What we're interested in is maximizing the average value of the sum of discounted rewards. By average value, I mean if you were to take your policy and try it out a thousand times or a 100,000 times or a million times, you get lots of different reward sequences like that and if you were to take the average over all of these different sequences of the sum of discounted rewards, then that's what we call the expected return.

In statistics, the term expected is just another way of saying average. But what this means is we want to maximize what we expect to get on average in terms of the sum of discounted rewards. The mathematical notation for this is to write this as E. E stands for expected value of $R_1 + \gamma * R_2 + \ldots$, and so on. The job of reinforcement learning algorithm is to choose a policy Pi to maximize the average or the expected sum of discounted rewards